TÀI LIỆU GIẢI THÍCH TOÀN BỘ MÃ NGUỒN FUNEWS MANAGEMENT SYSTEM

> ***Mục tiêu:*** *Diễn giải chi tiết 100% các biến, hàm và logic theo cách trực quan, mạch lạc, dễ hiểu nhất để bất kỳ ai dù chưa từng học lập trình sâu cũng có thể đọc hiểu và tự tay chỉnh sửa mã nguồn chính xác.*

MỤC LỤC

1. Khái niệm cốt lõi cần biết (#1-kh%C3%A1i-ni%E1%BB%87m-c%E1%BB%91t-l%C3%B5i-c%E1%BA%A7n-bi%E1%BA%BFt)
2. Cấu trúc tổng thể của dự án (#2-c%E1%BA%A5u-tr%C3%BAc-t%E1%BB%95ng-th%E1%BB%83-c%E1%BB%A7a-d%E1%BB%B1-%C3%A1n)
3. Tệp dữ liệu mẫu ban đầu: `src/data/initialData.js` (#3-t%E1%BB%87p-d%E1%BB%AF-li%E1%BB%87u-m%E1%BA%ABu-ban-%C4%91%E1%BA%A7u-srcdatainitialdatajs)
4. Bộ não điều khiển ứng dụng: `src/App.jsx` (#4-b%E1%BB%99-n%C3%A3o-%C4%91i%E1%BB%81u-khi%E1%BB%83n-%E1%BB%A9ng-d%E1%BB%A5ng-srcappjsx)
5. Các thành phần giao diện dùng chung (`src/components/`) (#5-c%C3%A1c-th%C3%A0nh-ph%E1%BA%A7n-giao-di%E1%BB%87n-d%C3%B9ng-chung-srccomponents)
    - Hộp thoại biểu mẫu: `Modal.jsx` (#h%E1%BB%99p-tho%E1%BA%A1i-bi%E1%BB%83u-m%E1%BA%ABu-modaljsx)
    - Hộp thoại xác nhận thao tác: `ConfirmDialog.jsx` (#h%E1%BB%99p-tho%E1%BA%A1i-x%C3%A1c-nh%E1%BA%ADn-thao-t%C3%A1c-confirmdialogjsx)
    - Thanh tiêu đề trên cùng: `Header.jsx` (#thanh-ti%C3%AAu-%C4%91%E1%BB%81-tr%C3%AAn-c%C3%B9ng-headerjsx)
    - Thanh điều hướng bên trái: `Sidebar.jsx` (#thanh-%C4%91i%E1%BB%81u-h%C6%B0%E1%BB%9Bng-b%C3%AAn-tr%C3%A1i-sidebarjsx)
    - Bố cục trang quản trị: `AdminLayout.jsx` (#b%E1%BB%91-c%E1%BB%A5c-trang-qu%E1%BA%A3n-tr%E1%BB%8B-adminlayoutjsx)
6. Chi tiết các trang chức năng (`src/pages/`) (#6-chi-ti%E1%BA%BFt-c%C3%A1c-trang-ch%E1%BB%A9c-n%C4%83ng-srcpages)
    - Trang Đăng nhập: `LoginPage.jsx` (#trang-%C4%91%C4%83ng-nh%E1%BA%ADp-loginpagejsx)
    - Trang Tổng quan: `DashboardPage.jsx` (#trang-t%E1%BB%95ng-quan-dashboardpagejsx)

    - Trang Quản lý Danh mục: `CategoryManagement.jsx` (#trang-qu%E1%BA%A3n-l%C3%BD-danh-m%E1%BB%A5c-categorymanagementjsx)
    - Trang Quản lý Tin tức: `NewsManagement.jsx` (#trang-qu%E1%BA%A3n-l%C3%BD-tin-t%E1%BB%A9c-newsmanagementjsx)
    - Trang Quản lý Người dùng: `UserManagement.jsx` (#trang-qu%E1%BA%A3n-l%C3%BD-ng%C6%B0%E1%BB%9Di-d%C3%B9ng-usermanagementjsx)
    - Trang Cài đặt hệ thống: `SettingsPage.jsx` (#trang-c%C3%A0i-%C4%91%E1%BA%B7t-h%E1%BB%87-th%E1%BB%91ng-settingspagejsx)
7. Điểm khởi động ứng dụng: `src/main.jsx` (#7-%C4%91i%E1%BB%83m-kh%E1%BB%9Fi-%C4%91%E1%BB%99ng-%E1%BB%A9ng-d%E1%BB%A5ng-srcmainjsx)
8. Cẩm nang hướng dẫn tự chỉnh sửa code (Thực hành thực tế) (#8-c%E1%BA%A9m-nang-h%C6%B0%E1%BB%9Bng-d%E1%BA%ABn-t%E1%BB%B1-ch%E1%BB%89nh-s%E1%BB%ADa-code)
9. Bảng tra cứu nhanh toàn bộ Biến & Hàm (#9-b%E1%BA%A3ng-tra-c%E1%BB%A9u-nhanh-to%C3%A0n-b%E1%BB%99-bi%E1%BA%BFn--h%C3%A0m)

---

1. Khái niệm cốt lõi cần biết

Để đọc hiểu mã nguồn ReactJS một cách dễ dàng, bạn chỉ cần nắm vững 5 khái niệm sau:

1. **Biến (Variable):** Một cái tên đại diện cho một dữ liệu cụ thể (số, chữ, danh sách). Khi cần dùng dữ liệu đó, ta gọi tên biến ra.
2. **State (Trạng thái - `useState`):** Là biến lưu dữ liệu động của giao diện. Điểm đặc biệt của State là: **khi giá trị của State thay đổi, React sẽ tự động vẽ lại giao diện tương ứng ngay lập tức**.
    - Cú pháp: `const [tenBien, setTenBien] = useState(giaTriBanDau);`
    - `tenBien`: Biến để đọc dữ liệu.
    - `setTenBien`: Hàm duy nhất dùng để gán giá trị mới cho biến đó.
3. **Hàm (Function):** Một tập hợp các câu lệnh được đóng gói dưới một cái tên. Khi có sự kiện xảy ra (như bấm nút, gõ phím), hàm sẽ được gọi để xử lý công việc.
4. **Props (Thuộc tính):** Dữ liệu hoặc hàm được component cha truyền xuống cho component con sử dụng.
5. **`localStorage` (Bộ nhớ trình duyệt):** Bộ nhớ lưu trữ dữ liệu trực tiếp trong trình duyệt web của máy tính. Dữ liệu lưu tại đây sẽ không bị mất đi kể cả khi bạn tải lại trang (F5) hoặc tắt trình duyệt.

2. Cấu trúc tổng thể của dự án

```
src/
├── data/
│   └── initialData.js            # Dữ liệu ban đầu + hàm đọc/ghi localStorage
├── components/
│   ├── common/
│   │   ├── Modal.jsx             # Cửa sổ nổi dùng cho Thêm / Sửa
│   │   └── ConfirmDialog.jsx     # Hộp thoại xác nhận trước khi Xóa
│   └── layout/
│       ├── Header.jsx            # Thanh ngang trên cùng (Logo, User, Logout)
│       ├── Sidebar.jsx           # Thanh dọc bên trái (5 mục Menu)
│       └── AdminLayout.jsx       # Khung bố cục chung cho trang quản trị
├── pages/
│   ├── LoginPage.jsx             # Giao diện đăng nhập
│   ├── DashboardPage.jsx         # Bảng điều khiển thống kê số liệu
│   ├── CategoryManagement.jsx    # Thêm, sửa, xóa, tìm kiếm Danh mục
│   ├── NewsManagement.jsx        # Thêm, sửa, xóa, tìm kiếm Bài báo
│   ├── UserManagement.jsx        # Thêm, sửa, xóa, tìm kiếm Tài khoản
│   └── SettingsPage.jsx          # Xem hồ sơ quản trị, reset dữ liệu
├── App.jsx                       # Thành phần trung tâm điều phối dữ liệu và màn hình
└── main.jsx                      # Điểm vào khởi chạy React trên trình duyệt
```

3. Tệp dữ liệu mẫu ban đầu: `src/data/initialData.js`

Tệp này chịu trách nhiệm cấp phát dữ liệu gốc ban đầu và cung cấp 2 hàm tiện ích để lưu/đọc dữ liệu từ bộ nhớ trình duyệt.

3.1. Các biến danh sách dữ liệu mẫu

- `defaultCategories` **(Mảng đối tượng):** Chứa các danh mục tin tức mặc định.
  - Mỗi phần tử gồm:
    - `id` (số): Mã định danh duy nhất của danh mục.
    - `name` (chuỗi): Tên danh mục (ví dụ: `"Technology"`, `"Campus Life"`).
    - `status` (số): `1` nghĩa là Đang kích hoạt (Active), `0` nghĩa là Tạm ẩn (Inactive).
- `defaultNews` **(Mảng đối tượng):** Chứa các bài viết tin tức mẫu.
  - Mỗi phần tử gồm:
    - `id` (số): Mã định danh duy nhất của bài viết.
    - `title` (chuỗi): Tiêu đề bài viết.
    - `content` (chuỗi): Nội dung chi tiết của bài viết.

        - `categoryId` (số): Mã danh mục liên kết với `defaultCategories`.
        - `createdBy` (chuỗi): Tên tác giả tạo bài (ví dụ: `"Admin"`, `"Staff"`).
        - `status` (số): `1` (Active - Hiển thị), `0` (Inactive - Ẩn).
- **`defaultUsers` (Mảng đối tượng):** Chứa các tài khoản người dùng mẫu để đăng nhập.
    - Mỗi phần tử gồm:
        - `id` (số): Mã định danh tài khoản.
        - `username` (chuỗi): Tên đăng nhập.
        - `password` (chuỗi): Mật khẩu đăng nhập.
        - `role` (số): Phân quyền: `1` là Quản trị viên (Admin), `2` là Nhân viên (Staff).
        - `status` (số): `1` là tài khoản hoạt động bình thường, `0` là tài khoản bị khóa.

3.2. Các hàm xử lý lưu trữ

Hàm `getStoredData(key, fallback)`

- **Mục đích:** Đọc dữ liệu từ `localStorage` lên bộ nhớ ứng dụng khi trang web vừa mở.
- **Tham số nhận vào:**
    - `key` (chuỗi): Tên khóa lưu trong `localStorage` (ví dụ: `"funews_categories"`).
    - `fallback` (bất kỳ): Dữ liệu mặc định trả về nếu trong `localStorage` chưa có gì hoặc xảy ra lỗi.
- **Quy trình hoạt động:**
    1. Gọi `localStorage.getItem(key)` để lấy chuỗi văn bản đã lưu.
    2. Nếu có dữ liệu: dùng `JSON.parse(item)` để chuyển chuỗi thành mảng/đối tượng Javascript và trả về.
    3. Nếu không có dữ liệu hoặc đọc lỗi: trả về giá trị `fallback`.

Hàm `setStoredData(key, value)`

- **Mục đích:** Lưu một mảng hoặc đối tượng vào `localStorage`.
- **Tham số nhận vào:**
    - `key` (chuỗi): Tên khóa muốn lưu.
    - `value` (bất kỳ): Dữ liệu muốn lưu trữ.
- **Quy trình hoạt động:**
    1. Dùng `JSON.stringify(value)` để biến đổi dữ liệu thành một chuỗi văn bản JSON.
    2. Gọi `localStorage.setItem(key, ...)` để ghi chuỗi đó vào bộ nhớ trình duyệt.

4. Bộ não điều khiển ứng dụng: `src/App.jsx`

File App.jsx () là thành phần gốc nắm giữ toàn bộ trạng thái dùng chung của ứng dụng.

4.1. Các biến State quản lý dữ liệu toàn cục

1. **`currentUser`:** Lưu thông tin tài khoản đang đăng nhập hiện tại.
   - Giá trị khởi tạo: Đọc từ `localStorage` với khóa `"funews_auth"`. Nếu chưa đăng nhập thì là `null`.
2. **`activeMenu`:** Lưu tên màn hình chức năng đang hiển thị.
   - Giá trị khởi tạo: `"dashboard"`. Các giá trị hợp lệ khác gồm: `"category"`, `"news"`, `"users"`, `"settings"`.
3. **`categories`:** Mảng danh sách toàn bộ các danh mục tin tức.
   - Khởi tạo: Đọc từ `"funews_categories"`, nếu trống thì lấy `defaultCategories`.
4. **`news`:** Mảng danh sách toàn bộ các bài viết.
   - Khởi tạo: Đọc từ `"funews_news"`, nếu trống thì lấy `defaultNews`.
5. **`users`:** Mảng danh sách toàn bộ tài khoản người dùng trong hệ thống.
   - Khởi tạo: Đọc từ `"funews_users"`, nếu trống thì lấy `defaultUsers`.

4.2. Các cơ chế đồng bộ tự động (`useEffect`)

- `useEffect` **categories (dòng 33-35):** Bất cứ khi nào mảng `categories` thay đổi (thêm, sửa, xóa danh mục), tự động gọi `setStoredData("funews_categories", categories)`.
- `useEffect` **news (dòng 37-39):** Bất cứ khi nào mảng `news` thay đổi, tự động ghi đè dữ liệu mới vào `"funews_news"`.
- `useEffect` **users (dòng 41-43):** Bất cứ khi nào mảng `users` thay đổi, tự động lưu vào `"funews_users"`.
- `useEffect` **currentUser (dòng 45-47):** Ghi nhớ trạng thái đăng nhập vào `"funews_auth"`.

4.3. Các hàm xử lý hành động

Hàm `handleLogin(user)`

- **Khi nào chạy:** Khi người dùng nhập đúng thông tin ở màn hình đăng nhập và bấm "Sign In".
- **Đầu vào:** Đối tượng `user` chứa thông tin tài khoản hợp lệ.
- **Xử lý:**
  1. `setCurrentUser(user)`: Lưu tài khoản này vào bộ nhớ ứng dụng.
  2. `setActiveMenu("dashboard")`: Chuyển màn hình về trang Tổng quan.

Hàm `handleLogout()`

- **Khi nào chạy:** Khi bấm nút "Logout" trên thanh Header.
- **Xử lý:**
  1. `setCurrentUser(null)`: Xóa tài khoản đang đăng nhập trong state.
  2. `localStorage.removeItem("funews_auth")`: Xóa phiên đăng nhập khỏi bộ nhớ trình duyệt.
  3. Khi `currentUser` biến thành `null`, giao diện tự động chuyển về trang Đăng nhập.

Hàm `handleResetData()`

- **Khi nào chạy:** Khi người dùng bấm nút "Reset to Default Seed Data" trong trang Cài đặt.
- **Xử lý:** Ghi đè lại cả 3 danh sách `categories`, `news`, `users` về đúng dữ liệu gốc ban đầu (`defaultCategories`, `defaultNews`, `defaultUsers`) cả trong bộ nhớ RAM lẫn `localStorage`.

4.4. Cơ chế hiển thị màn hình (Render Flow)

- **Nếu `!currentUser` (chưa đăng nhập):** Trả về `<LoginPage onLogin={handleLogin} users={users} />`.
- **Nếu đã đăng nhập:** Hiển thị `<AdminLayout>` bọc bên ngoài. Bên trong sẽ dựa vào biến `activeMenu` để hiển thị trang tương ứng:
  - `activeMenu === "dashboard"` → `<DashboardPage />`
  - `activeMenu === "category"` → `<CategoryManagement />`
  - `activeMenu === "news"` → `<NewsManagement />`
  - `activeMenu === "users"` → `<UserManagement />`

- `activeMenu === "settings"` → `<SettingsPage />`

---

5. Các thành phần giao diện dùng chung (`src/components/`)

Hộp thoại biểu mẫu: `Modal.jsx`

File: Modal.jsx ()

- **Vai trò:** Hiển thị một khung cửa sổ nổi trên màn hình có lớp nền tối mờ phía sau. Dùng để chứa form nhập liệu cho cả việc Thêm mới và Chỉnh sửa.
- **Các Props nhận vào:**
  - `isOpen` (boolean: `true` / `false`): Quyết định xem hộp thoại có hiển thị hay không. Nếu `false`, hàm trả về `null` (không vẽ gì).
  - `title` (chuỗi): Tiêu đề hiển thị trên đỉnh của hộp thoại (ví dụ: `"Create Category"`).
  - `onClose` (hàm): Hàm được gọi khi người dùng bấm dấu `×` hoặc click ra ngoài lớp nền mờ.
  - `children` (phần tử JSX): Toàn bộ nội dung biểu mẫu bên trong do trang cha truyền vào.
- **Chi tiết kỹ thuật:** Thẻ chứa hộp thoại có lệnh `onClick={(e) => e.stopPropagation()}` để khi người dùng nhấn chuột vào bên trong biểu mẫu, sự kiện không bị truyền ra ngoài lớp nền làm đóng hộp thoại ngoài ý muốn.

Hộp thoại xác nhận thao tác: `ConfirmDialog.jsx`

File: ConfirmDialog.jsx ()

- **Vai trò:** Ngăn chặn việc người dùng vô tình bấm nhầm nút Xóa. Luôn yêu cầu người dùng xác nhận lại trước khi xóa dữ liệu thật sự.
- **Các Props nhận vào:**
  - `isOpen` (boolean): Trạng thái đóng/mở của hộp thoại.
  - `title` (chuỗi): Tiêu đề cảnh báo (mặc định là `"Confirm Action"`).
  - `message` (chuỗi): Lời nhắn giải thích cụ thể đối tượng nào sắp bị xóa.
  - `onConfirm` (hàm): Hàm được gọi khi người dùng bấm nút đỏ `"Delete"`.
  - `onCancel` (hàm): Hàm được gọi khi người dùng bấm nút xám `"Cancel"` hoặc nút `×`.

Thanh tiêu đề trên cùng: `Header.jsx`

File: Header.jsx ()

- **Vai trò:** Thanh điều hướng cố định trên cùng màn hình quản trị.
- **Các Props nhận vào:**
  - `currentUser`: Đối tượng người dùng hiện tại để lấy thuộc tính `currentUser?.username` hiển thị lên huy hiệu người dùng. Nếu không có tên, mặc định hiển thị `"Admin"`.
  - `onLogout`: Hàm truyền từ `App.jsx`, được gắn vào sự kiện `onClick` của nút "Logout".

Thanh điều hướng bên trái: `Sidebar.jsx`

File: Sidebar.jsx ()

- **Vai trò:** Cung cấp menu chuyển đổi giữa 5 màn hình quản trị.
- **Biến nội bộ:**
  - `menuItems` (mảng đối tượng): Danh sách các mục menu gồm `{ key, label }`:
    - `dashboard` → Dashboard
    - `category` → Category
    - `news` → News
    - `users` → Users
    - `settings` → Settings
- **Các Props nhận vào:**
  - `activeMenu` (chuỗi): Khóa của mục đang mở để thêm class CSS `active` làm sáng nút tương ứng.
  - `onSelectMenu` (hàm): Hàm được gọi khi bấm vào một mục menu, truyền `item.key` ngược lên cho `App.jsx` đổi trang.

Bố cục trang quản trị: `AdminLayout.jsx`

File: AdminLayout.jsx ()

- **Vai trò:** Khung mẫu tổng thể.

- **Cấu trúc:** Ghép `Header` ở trên cùng, phía dưới chia làm 2 phần: `Sidebar` nằm cố định bên trái, và vùng nội dung chính `<main className="admin-content">{children}</main>` nằm bên phải hiển thị trang đang chọn.

---

6. Chi tiết các trang chức năng (`src/pages/`)

Trang Đăng nhập: `LoginPage.jsx`

File: LoginPage.jsx ()

1. Các biến State:

- `username` (chuỗi): Giá trị văn bản người dùng đang gõ trong ô Tên đăng nhập.
- `password` (chuỗi): Giá trị văn bản người dùng đang gõ trong ô Mật khẩu.
- `error` (chuỗi): Thông báo lỗi màu đỏ khi đăng nhập thất bại. Nếu rỗng `""` thì không hiển thị cảnh báo.

2. Các Props nhận vào:

- `onLogin`: Hàm kích hoạt đăng nhập thành công do `App.jsx` truyền xuống.
- `users`: Danh sách toàn bộ tài khoản để kiểm tra tính hợp lệ khi đăng nhập.

3. Hàm `handleSubmit(e)`:

- **Khi nào chạy:** Khi người dùng bấm nút "Sign In" hoặc nhấn phím Enter trên form đăng nhập.
- **Các bước xử lý logic:**
  1. `e.preventDefault()`: Ngăn trình duyệt tự tải lại trang.
  2. `trimmedUser = username.trim()`, `trimmedPass = password.trim()`: Loại bỏ các khoảng trắng thừa ở hai đầu.
  3. **Kiểm tra rỗng:** Nếu thiếu một trong hai trường, gán `error = "Please enter both username and password."` và dừng lại.
  4. **Kiểm tra tài khoản quản trị Admin mặc định:**
     - Nếu `trimmedUser === "Admin"` và `trimmedPass === "Admin"`:
     - Tìm xem trong `users` có tài khoản Admin chưa, nếu có thì lấy, nếu chưa có thì dùng tài khoản mặc định `{ id: 1, username: "Admin", role: 1, status: 1 }`.
     - Gọi `onLogin(adminUser)` và hoàn tất đăng nhập.

5. **Kiểm tra danh sách người dùng thông thường:**
    - Tìm kiếm người dùng có tên trùng khớp (không phân biệt hoa thường) và mật khẩu khớp tuyệt đối.
    - Nếu tìm thấy:
        - Kiểm tra `matchedUser.status === 0`: Nếu tài khoản bị khóa, báo lỗi: `"This account is currently inactive."` và không cho vào.
        - Nếu tài khoản hoạt động: Xóa thông báo lỗi và gọi `onLogin(matchedUser)`.
    - Nếu không tìm thấy: Báo lỗi `"Invalid username or password."`.

Trang Tổng quan: `DashboardPage.jsx`

File: DashboardPage.jsx ()

1. Các Props nhận vào:

- `categories` (mảng danh mục), `news` (mảng bài viết), `users` (mảng người dùng).

2. Các biến tính toán tự động (Derived State):

- `activeCategoriesCount`: Lọc số danh mục có `status === 1`.
- `inactiveCategoriesCount`: Lọc số danh mục có `status === 0`.
- `activeNewsCount`: Lọc số bài viết có `status === 1`.
- `inactiveNewsCount`: Lọc số bài viết có `status === 0`.
- `adminUsersCount`: Lọc số người dùng có vai trò quản trị `role === 1`.
- `staffUsersCount`: Lọc số nhân viên có `role === 2`.

3. Hàm `getCategoryName(categoryId)`:

- **Mục đích:** Nhận vào mã số `categoryId` của bài viết và tìm tên danh mục tương ứng trong mảng `categories`. Nếu không tìm thấy, trả về chữ `"Uncategorized"`.

4. Nội dung hiển thị:

- Khối thẻ thống kê gồm 3 thẻ: Tổng số Danh mục, Tổng số Bài viết, Tổng số Tài khoản (kèm chi tiết số lượng kích hoạt/không kích hoạt).
- Bảng danh sách 5 bài báo mới nhất được lấy qua lệnh `news.slice(0, 5)`.

Trang Quản lý Danh mục: `CategoryManagement.jsx`

File: CategoryManagement.jsx ()

1. Các biến State:

- `searchTerm` (chuỗi): Từ khóa tìm kiếm do người dùng gõ vào ô tìm kiếm.
- `isModalOpen` (boolean): Điều khiển hiển thị hộp thoại Thêm/Sửa.
- `modalMode` (chuỗi): `"create"` khi đang thêm mới; `"update"` khi đang sửa.
- `selectedCategory` (đối tượng): Danh mục đang được chọn để sửa (chứa dữ liệu cũ).
- `formData` (đối tượng): Dữ liệu nhập trên form `{ name: "", status: 1 }`.
- `formError` (chuỗi): Thông báo lỗi khi người dùng để trống tên danh mục.
- `deleteTarget` (đối tượng): Danh mục đang được chọn để xóa.
- `isConfirmOpen` (boolean): Điều khiển hiển thị hộp thoại xác nhận xóa.
- `relationError` (chuỗi): Cảnh báo khi danh mục đang bị ràng buộc bởi các bài viết liên quan.

2. Các biến tính toán:

- `normalizedSearch`: Từ khóa tìm kiếm đã cắt khoảng trắng và đưa về chữ thường.
- `displayedCategories`: Danh sách danh mục đã lọc theo `normalizedSearch` để hiển thị trên bảng.

3. Toàn bộ các hàm xử lý:

- **`handleOpenCreate()`**:
  - Đặt `modalMode = "create"`, xóa `selectedCategory = null`.
  - Đặt form về mặc định `{ name: "", status: 1 }` và mở modal (`isModalOpen = true`).
- **`handleOpenEdit(category)`**:
  - Đặt `modalMode = "update"`, lưu danh mục đang chọn vào `selectedCategory`.
  - Đổ dữ liệu cũ vào form `{ name: category.name, status: category.status }` và mở modal.
- **`handleCloseModal()`**:
  - Đóng modal và dọn sạch dữ liệu tạm trên form.
- **`handleFormSubmit(e)`**:

    - Ngăn reload trang. Kiểm tra nếu `formData.name.trim()` bị rỗng thì báo lỗi `"Category name is required."`.
    - **Nếu tạo mới (`modalMode === "create"`):**
        - Tính ID tiếp theo: `Math.max(...categories.map(c => c.id)) + 1` (nếu mảng rỗng thì lấy `1`).
        - Tạo đối tượng danh mục mới và thêm vào danh sách: `setCategories([...categories, newCategory])`.
    - **Nếu chỉnh sửa (`modalMode === "update"`):**
        - Dùng `categories.map()` tìm danh mục có `id === selectedCategory.id` để cập nhật tên và trạng thái mới.
    - Đóng modal.
- **`handleOpenDelete(category)` (Bảo toàn dữ liệu):**
    - Kiểm tra xem có bài báo nào đang sử dụng danh mục này không: `news.filter(n => n.categoryId === category.id)`.
    - Nếu có: Không cho xóa, hiển thị thông báo lỗi màu đỏ trong 5 giây và dừng lại.
    - Nếu không có bài báo nào phụ thuộc: Lưu danh mục vào `deleteTarget` và mở hộp thoại xác nhận `ConfirmDialog`.
- **`handleConfirmDelete()`:**
    - Lọc bỏ danh mục có ID trùng với `deleteTarget.id`: `setCategories(categories.filter(c => c.id !== deleteTarget.id))`.
    - Đóng hộp thoại xác nhận.
- **`handleCancelDelete()`:** Hủy bỏ thao tác xóa, đóng hộp thoại.

Trang Quản lý Tin tức: `NewsManagement.jsx`

File: NewsManagement.jsx ()

1. Các biến State:

- `searchTerm`: Từ khóa tìm kiếm bài viết (tìm cả theo tiêu đề lẫn nội dung).
- `isModalOpen`: Điều khiển hộp thoại thêm/sửa bài viết.
- `modalMode`: Chế độ `"create"` hoặc `"update"`.
- `selectedArticle`: Bài viết đang được chọn chỉnh sửa.
- `formData`: Chứa các trường dữ liệu của bài báo:
    - `title` (tiêu đề)

    - `content` (nội dung)
    - `categoryId` (mã danh mục)
    - `createdBy` (tên người tạo)
    - `status` (trạng thái: 1 - Hiển thị, 0 - Ẩn)
- `formError`: Báo lỗi nếu thiếu tiêu đề, nội dung hoặc danh mục không hợp lệ.
- `deleteTarget`: Bài viết đang chọn xóa.
- `isConfirmOpen`: Hộp thoại xác nhận xóa bài viết.

2. Các hàm xử lý:

- **`getCategoryName(catId)`**: Tìm tên danh mục từ ID để hiển thị lên bảng.
- **`handleOpenCreate()`**: Khởi tạo form trắng với `createdBy` lấy từ tài khoản đang đăng nhập `currentUser?.username`.
- **`handleOpenEdit(article)`**: Nạp toàn bộ dữ liệu của bài viết cần sửa vào form và mở modal.
- **`handleCloseModal()`**: Đóng form và làm trống dữ liệu tạm.
- **`handleFormSubmit(e)`**:
    - Kiểm tra bắt buộc: Tiêu đề và nội dung không được để trống.
    - Kiểm tra danh mục: Hệ thống phải có ít nhất 1 danh mục và danh mục được chọn phải tồn tại trong mảng `categories`.
    - Thực hiện thêm mới với ID tự tăng hoặc cập nhật bài viết cũ qua phương thức `map()`.
    - Cập nhật state `news` qua `setNews` và đóng modal.
- **`handleOpenDelete(article)`**: Gán bài viết vào `deleteTarget` và mở hộp thoại xác nhận.
- **`handleConfirmDelete()`**: Loại bỏ bài viết ra khỏi mảng `news` thông qua phương thức `filter()`.
- **`handleCancelDelete()`**: Đóng hộp thoại xác nhận và hủy thao tác.

Trang Quản lý Người dùng: `UserManagement.jsx`

File: UserManagement.jsx ()

1. Các biến State:

- `searchTerm`: Từ khóa tìm kiếm tài khoản theo tên đăng nhập (`username`).
- `isModalOpen`: Trạng thái mở modal biểu mẫu.
- `modalMode`: `"create"` hoặc `"update"`.

- `selectedUser`: Người dùng đang được chọn chỉnh sửa.
- `formData`: Chứa `{ username, password, role, status }`.
- `formError`: Báo lỗi nếu để trống hoặc trùng tên tài khoản.
- `deleteTarget`: Tài khoản được chọn xóa.
- `isConfirmOpen`: Hộp thoại xác nhận xóa tài khoản.

2. Các hàm và quy tắc kiểm tra logic quan trọng:

- **`handleFormSubmit(e)`:**
  - Kiểm tra không để trống Username và Password.
  - **Kiểm tra chống trùng lặp tên đăng nhập:**

    ```
    const duplicateUser = users.find(
      (u) =>
        (u.username || "").toLowerCase() === trimmedUsername.toLowerCase() &&
        (modalMode === "create" || u.id !== selectedUser?.id)
    );
    ```

    Nếu tên đăng nhập đã được người khác sử dụng, chặn lưu và báo lỗi: `"Username is already taken. Please choose another."`.
  - Tạo mới với ID tự tăng hoặc cập nhật tài khoản cũ, sau đó đóng modal.
- **Cơ chế chống tự xóa chính mình:**
  - Trên bảng dữ liệu, tài khoản đang đăng nhập được gắn thêm huy hiệu `(You)`.
  - Nút **Delete** của tài khoản này bị vô hiệu hóa bằng thuộc tính `disabled` khi `currentUser.username === user.username`.
- **`handleConfirmDelete()` / `handleCancelDelete()`:** Tiến hành xóa hoặc hủy xóa tài khoản tương tự như danh mục và bài viết.

Trang Cài đặt hệ thống: `SettingsPage.jsx`

File: SettingsPage.jsx ()

1. Các biến State:

- `systemName`: Tên hệ thống hiển thị trong biểu mẫu cấu hình (mặc định `"FUNewsManagementSystem"`).
- `contactEmail`: Email quản trị viên liên hệ (mặc định `"admin@funews.edu.vn"`).

- `saveSuccess` (boolean): Cờ thông báo lưu thành công. Khi bật `true`, một thông báo màu xanh lá sẽ hiện ra trong 3 giây.
- `isResetConfirmOpen` (boolean): Hộp thoại xác nhận khôi phục dữ liệu gốc.

2. Các hàm xử lý:

- **`handleSaveSettings(e)`**: Ngăn reload trang, bật thông báo `setSaveSuccess(true)` và tự động tắt sau 3 giây bằng `setTimeout`.
- **`handleConfirmReset()`**: Gọi hàm `onResetData()` do `App.jsx` truyền xuống để xóa toàn bộ dữ liệu người dùng đã chỉnh sửa, đưa hệ thống về dữ liệu mẫu ban đầu, sau đó bật thông báo thành công.

---

7. Điểm khởi động ứng dụng: `src/main.jsx`

File main.jsx () là nơi ứng dụng được nạp vào trình duyệt:

1. `document.getElementById('root')`: Tìm thẻ `<div>` có `id="root"` trong file `index.html`.
2. `createRoot(...)`: Tạo gốc ứng dụng của React 18.
3. `.render(<StrictMode><App /></StrictMode>)`: Kích hoạt việc vẽ giao diện toàn bộ ứng dụng từ component `<App />`.

---

8. Cẩm nang hướng dẫn tự chỉnh sửa code

Dưới đây là các tình huống thực tế thường gặp và hướng dẫn từng bước cụ thể để bạn tự chỉnh sửa mã nguồn:

Tình huống 1: Thay đổi thông tin tài khoản đăng nhập Admin mặc định

- **Mục tiêu:** Đổi mật khẩu Admin từ `"Admin"` thành `"MatKhauBaoMat2026"`.
- **Các bước thực hiện:**
  1. Mở file LoginPage.jsx (), tìm đến dòng 19:

```
// Sửa dòng này:
if (trimmedUser === "Admin" && trimmedPass === "Admin") {

// Thành:
if (trimmedUser === "Admin" && trimmedPass === "MatKhauBaoMat2026") {
```

2. Mở file initialData.js (), tìm đến dòng 44:

```
// Sửa password của Admin:
{ id: 1, username: "Admin", password: "MatKhauBaoMat2026", role: 1, status: 1 },
```

3. Mở trang web, vào mục **Settings**, bấm nút **Reset to Default Seed Data** để nạp mật khẩu mới vào `localStorage`.

Tình huống 2: Đổi tên hiển thị của hệ thống trên thanh Header

- **Mục tiêu:** Đổi tên `"FUNewsManagementSystem"` thành `"Cổng Thông Tin Đại Học FPT"`.
- **Các bước thực hiện:**
  1. Mở file Header.jsx (), tìm dòng 8:

```
// Sửa dòng:
<span className="header-title">FUNewsManagementSystem</span>

// Thành:
<span className="header-title">Cổng Thông Tin Đại Học FPT</span>
```

  2. Mở file LoginPage.jsx (), tìm dòng 52 và sửa tương tự cho màn hình đăng nhập.

Tình huống 3: Bổ sung thêm một danh mục tin tức mặc định mới

- **Mục tiêu:** Thêm danh mục `"Tuyển sinh & Học bổng"` (ID: 5) vào hệ thống.
- **Các bước thực hiện:**
  1. Mở file initialData.js (), tìm mảng `defaultCategories`:

```
export const defaultCategories = [
  { id: 1, name: "Technology", status: 1 },
  { id: 2, name: "Campus Life", status: 1 },
  { id: 3, name: "Academic", status: 1 },
  { id: 4, name: "Events", status: 0 },
  { id: 5, name: "Tuyển sinh & Học bổng", status: 1 } // <-- Thêm dòng này
];
```

  2. Lưu file và thực hiện bấm nút Reset trong trang Settings để nạp lại dữ liệu.

Tình huống 4: Thêm quy tắc mật khẩu phải có ít nhất 6 ký tự khi tạo người dùng

- **Mục tiêu:** Không cho phép tạo tài khoản nếu mật khẩu ngắn hơn 6 ký tự.
- **Các bước thực hiện:**
  1. Mở file UserManagement.jsx, tìm hàm `handleFormSubmit` ở khoảng dòng 66:

```
if (!trimmedUsername || !trimmedPassword) {
  setFormError("Both Username and Password are required.");
  return;
}

// THÊM ĐOẠN KIỂM TRA MỚI VÀO ĐÂY:
if (trimmedPassword.length < 6) {
  setFormError("Mật khẩu phải chứa ít nhất 6 ký tự.");
  return;
}
```

  2. Bây giờ khi bạn tạo tài khoản với mật khẩu quá ngắn, hệ thống sẽ tự động hiện thông báo lỗi và ngăn thao tác tạo.

Tình huống 5: Việt hóa các nút thao tác trên bảng dữ liệu (Edit / Delete)

- **Mục tiêu:** Chuyển đổi các chữ tiếng Anh `"Edit"` thành `"Sửa"` và `"Delete"` thành `"Xóa"`.
- **Các bước thực hiện:**
  1. Mở file CategoryManagement.jsx, tìm các nút ở dòng 177 và 184:
     - Sửa `<button ...>Edit</button>` thành `<button ...>Sửa</button>`.
     - Sửa `<button ...>Delete</button>` thành `<button ...>Xóa</button>`.
  2. Làm tương tự cho file NewsManagement.jsx (dòng 233, 240) và UserManagement.jsx (dòng 220, 233).

9. Bảng tra cứu nhanh toàn bộ Biến & Hàm

Tên File	Tên Biến / Hàm	Kiểu	Ý nghĩa và Chức năng chi tiết
`initialData.js`	`defaultCategories`	Mảng dữ liệu	Danh sách các danh mục tin tức ban đầu
`initialData.js`	`defaultNews`	Mảng dữ liệu	Danh sách các bài viết mẫu ban đầu
`initialData.js`	`defaultUsers`	Mảng dữ liệu	Danh sách tài khoản Admin và Staff mẫu
`initialData.js`	`getStoredData(key, fallback)`	Hàm	Đọc dữ liệu từ `localStorage`, nếu lỗi trả về giá trị dự phòng
`initialData.js`	`setStoredData(key, value)`	Hàm	Ghi dữ liệu đã chuyển đổi thành chuỗi JSON vào `localStorage`
`App.jsx`	`currentUser`	State	Chứa thông tin người dùng đang đăng nhập (`null` nếu chưa đăng nhập)
`App.jsx`	`activeMenu`	State	Chuỗi định danh trang hiện tại (`"dashboard"`, `"category"`,...)
`App.jsx`	`categories`, `news`, `users`	State	Mảng dữ liệu trung tâm của toàn bộ ứng dụng

Tên File	Tên Biến / Hàm	Kiểu	Ý nghĩa và Chức năng chi tiết
`App.jsx`	`handleLogin(user)`	Hàm	Cập nhật người dùng đăng nhập và chuyển hướng sang Dashboard
`App.jsx`	`handleLogout()`	Hàm	Xóa session trong State và `localStorage`, quay lại màn hình Login
`App.jsx`	`handleResetData()`	Hàm	Khôi phục toàn bộ dữ liệu hệ thống về seed data ban đầu
`Modal.jsx`	`isOpen`, `title`, `onClose`, `children`	Props	Các thuộc tính cấu hình và nội dung của cửa sổ popup nổi
`ConfirmDialog.jsx`	`isOpen`, `title`, `message`, `onConfirm`, `onCancel`	Props	Cấu hình hộp thoại xác nhận hủy/xóa
`Header.jsx`	`currentUser`, `onLogout`	Props	Thông tin user và hàm đăng xuất gắn trên thanh tiêu đề
`Sidebar.jsx`	`menuItems`, `activeMenu`, `onSelectMenu`	Mảng / Props	Quản lý danh sách và sự kiện chuyển đổi 5 menu điều hướng
`LoginPage.jsx`	`username`, `password`, `error`	State	Lưu dữ liệu gõ phím và thông báo lỗi trên form đăng nhập
`LoginPage.jsx`	`handleSubmit(e)`	Hàm	Kiểm tra dữ liệu đăng nhập, hỗ

Tên File	Tên Biến / Hàm	Kiểu	Ý nghĩa và Chức năng chi tiết
			trợ Admin/Admin và tài khoản Staff
`DashboardPage.jsx`	`activeCategoriesCount`, `activeNewsCount`,...	Biến tính toán	Các chỉ số thống kê số lượng dữ liệu kích hoạt/ẩn
`DashboardPage.jsx`	`getCategoryName(id)`	Hàm	Tra cứu tên danh mục từ mã ID tương ứng
`CategoryManagement.jsx`	`searchTerm`, `formData`, `selectedCategory`, `deleteTarget`	State	Quản lý trạng thái tìm kiếm, nhập liệu và đối tượng sửa/xóa danh mục
`CategoryManagement.jsx`	`handleFormSubmit(e)`	Hàm	Thực hiện Thêm mới hoặc Cập nhật danh mục kèm validation
`CategoryManagement.jsx`	`handleOpenDelete(category)`	Hàm	Kiểm tra toàn vẹn dữ liệu: chặn xóa danh mục nếu đang có bài báo sử dụng
`NewsManagement.jsx`	`searchTerm`, `formData`, `selectedArticle`, `deleteTarget`	State	Quản lý biểu mẫu bài viết, danh mục liên kết và đối tượng xóa
`NewsManagement.jsx`	`handleFormSubmit(e)`	Hàm	Kiểm tra dữ liệu và lưu bài viết vào mảng `news`

Tên File	Tên Biến / Hàm	Kiểu	Ý nghĩa và Chức năng chi tiết
`UserManagement.jsx`	`searchTerm`, `formData`, `selectedUser`, `deleteTarget`	State	Quản lý thông tin tài khoản, phân quyền Role 1/2 và trạng thái
`UserManagement.jsx`	`handleFormSubmit(e)`	Hàm	Kiểm tra chống trùng lặp `username` và lưu tài khoản
`SettingsPage.jsx`	`systemName`, `contactEmail`, `saveSuccess`	State	Lưu thông tin cấu hình hệ thống và cờ thông báo lưu thành công
`SettingsPage.jsx`	`handleSaveSettings(e)`	Hàm	Xử lý lưu cấu hình và tự tắt thông báo thành công sau 3 giây
`SettingsPage.jsx`	`handleConfirmReset()`	Hàm	Xác nhận xóa sạch dữ liệu sửa đổi, nạp lại dữ liệu hạt giống

---

> 💡 ***Lời khuyên khi chạy thử nghiệm:*** *Sau khi chỉnh sửa bất kỳ đoạn code nào, bạn hãy mở terminal tại thư mục dự án và chạy lệnh:*
>
> ```
> npm run dev
> ```
>
> *Trình duyệt sẽ tự động cập nhật ngay lập tức những thay đổi của bạn mà không cần phải khởi động lại!*